

Apuntes Semana 10 - Clase del Martes

Mario Adrián Sánchez Ponce, 2022167079, Instituto Tecnológico de Costa Rica

I. RESPUESTAS DEL QUIZ

I-A. Describa que es una red totalmente conectada incluyendo sus 3 capas (Fully-connected)

Consiste en una red compleja donde se pasa por funciones no lineales para llegar a la salida. Se compone por la capa de entrada (Que contiene los features), la capa intermedia (una o varias funciones de activación) y la capa de salida (que contiene la función de activación final para su clasificación)-

I-B. ¿Qué es un one-hot vector y como me ayuda a resolver problemas de regresión multinomial

Se trata de una codificación donde se enciende un bit correspondiente a la entrada registrada. Se comparan los vectores con el fin de tener una salida de múltiple regresiones

I-C. Mencione tres técnicas de augmentation de datos para señales de audio propuestas en Spec Augment

- Time Warping - Frequency Masking - Time Masking

I-D. ¿Por qué preferimos resolver problemas de clasificación binaria con una red neuronal de mínimo una capa oculta antes que una regresión logística

Se utiliza la clasificación binaria pues permite la resolución de problemas no lineales, lo que significa que aborda un set de problemas más complejo contrario a lo limitado que es con simple regresión logística.

II. REPASO - REDES NEURONALES

II-A. Perceptrón

Buscaba mejorar la regresión logística, el problema fue que solo funcionaba con problemas lineales. Esto provocó el invierno de la inteligencia artificial en los años 70, pues no se podían solucionar problemas complejos como el XOR. Fue sino hasta la llegada de las redes neuronales que hubo un avance significativo en esta tecnología.

II-B. Redes Neuronales

Tienen una neurona con una intensidad específica que pasa por una función de activación. Por ejemplo, en regresión logística se tiende a utilizar la función sigmoide.

En un perceptrón multicapa se trata de una composición de funciones, la siguiente función dependiendo de la anterior.

$$h^{(0)} = \text{sigmoid} \left(XW^{(0)} + b^{(0)} \right)$$

$$h^{(1)} = \text{sigmoid} \left(h^{(0)}W^{(0)} + b^{(0)} \right)$$

$$h^{(n)} = \text{sigmoid} \left(h^{(n-1)}W^{(0)} + b^{(0)} \right)$$

De momento se tenían probabilidades independientes. Sin poder una distribución de probabilidad cuya suma diera 1 como resultado, el cual es lo que se busca. Tanto la suma de 1 como el one-hot vector.

En la capa de salida no es recomendable utilizar sigmoide por esto mismo. Se necesita una función de pérdida y una función de activación que sea capaz de brindarnos una distribución de probabilidad.

En la función de pérdida se aplica en base a la derivada de función de costos, esta en base en cada neurona. Se optimizan los parámetros w y b de cada una de las neuronas para optimizar L .

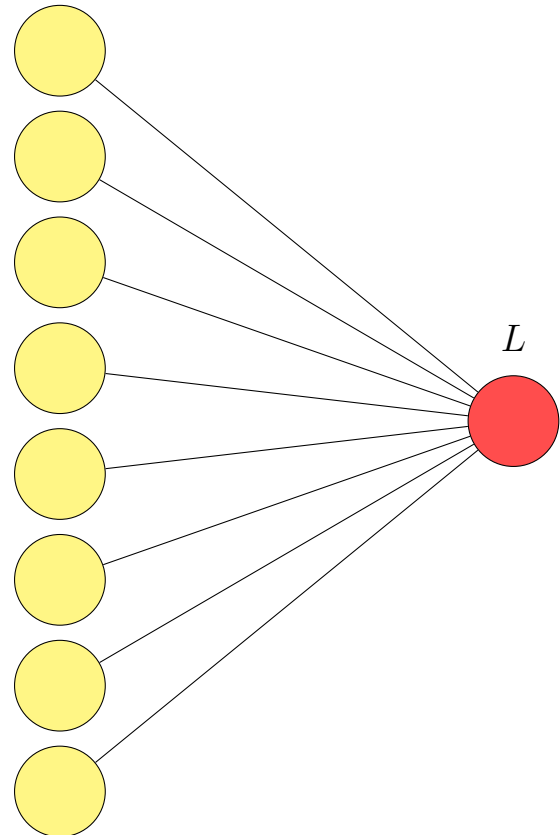
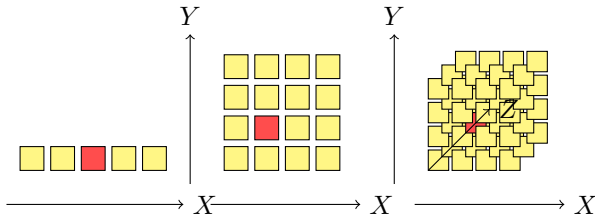


Figura 1. Función de pérdida en base a las neuronas de la capa anterior

II-C. Distribución de jerarquía - Maldición de dimensionalidad

Si se escoge elementos demasiado grandes aumenta la función de coste. Por ello se busca mantener la dimensionalidad baja.



Entre más dimensiones se añaden, mayor la dificultad de identificar un solo elemento. Para ello se mantiene una búsqueda de comportamiento jerárquico que va desde la regresión lineal hasta el perceptrón multicapa.

Con esto se genera una ganancia exponencial en cuanto al aprendizaje pues se toma una ruta desde lo más simple como polinomios, hasta problemas de mayor complejidad como face recognition. Por medio de embedding se puede mapear la imagen en un vector de 512 bits que vigila el valor semántico de una imagen.

II-D. Convolutional Neural Networks (CNN)

Consiste en un pequeño kernel que se desliza a través de la imagen para obtener las características y cada vez aumenta en tamaño y complejidad hasta que sea capaz de proyectar la imagen capturada.

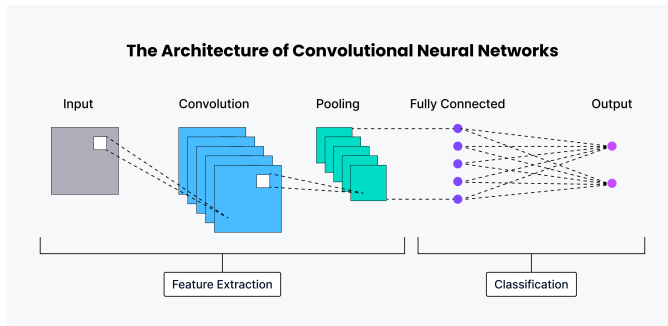


Figura 2. Arquitectura de CNN [1]

III. FUNCIÓN DE ACTIVACIÓN - REPASO

Agrega comportamiento a la red. Puede acotar las salidas en cada capa para analizar y proyectar el resultado de la red.

III-A. Función Lineal

Si bien funciona en problemas simples, el problema principal se encuentra en el gradiente, pues la derivada de este es un número fijo

$$y = mx + b \quad (1)$$

$$\frac{d}{dx}(mx + b) = m \quad (2)$$

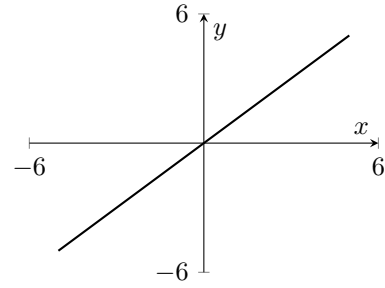


Figura 3. Función lineal. Si lo amerita

III-B. Función sigmoide

Acota entre 0 y 1. Siempre es positiva. Permite el descenso sobre la función de costo. El problema se encuentra en los extremos donde la gradiente es casi 0 constante, por lo que casi no avanza en dicha función de costo. Eso hace que el gradiente sea diminuto y provoca una falta de optimización

$$\sigma(x) = \frac{1}{1 + e^{-x}} \quad (3)$$

$$\frac{d\sigma(x)}{dx} = \sigma(x)(1 - \sigma(x)) \quad (4)$$

...

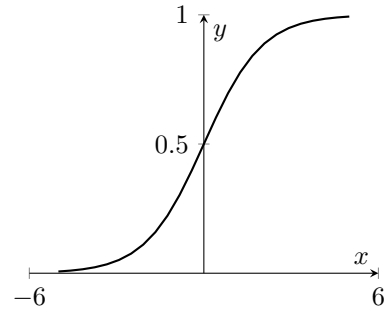


Figura 4. Función sigmoide.

III-C. Función tangente hiperbólica

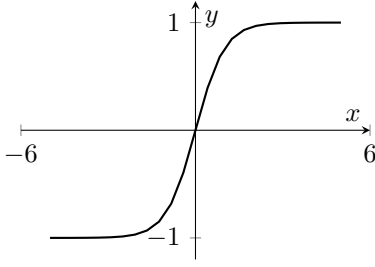
Mismos problemas que la función sigmoide.

$$\tanh(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}} \quad (5)$$

$$\frac{d}{dx} \tanh(x) = 1 - \tanh^2(x) \quad (6)$$

III-D. Función ReLU

Se mantiene en 0 mientras x sea inferior. Cuando supera 0, esta función se vuelve creciente. Es eficiente para Deep Learning y Computer Vision. El problema es que no es del todo derivable, ya que queda indefinido en 0. Esto significa que cuando el gradiente llega a 0 los pesos no son actualizados, terminando en neuronas muertas.

Figura 5. Función tangente hiperbólica. Los valores se acotan en $[-1, 1]$

$$\text{ReLU}(x) = \max(0, x) \quad (7)$$

$$\frac{d}{dx}\text{ReLU}(x) = \begin{cases} 0, & x < 0 \\ 1, & x > 0 \end{cases} \quad (8)$$

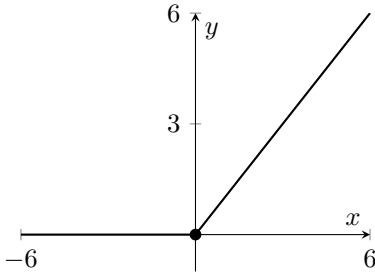


Figura 6. Función ReLU

III-E. Leaky ReLU

Alternativa que busca solucionar el problema de las neuronas muertas. Esto lo hace por medio de la asignación de una constante al mínimp permitido. El nuevo problema surge en la arbitrariedad al momento de seleccionar esta constante, pues al ser una decisión sin ninguna base (más que todo un estándar), puede que no sea la constante correcta para el caso.

$$\text{LeakyReLU}(x) = \begin{cases} x, & x \geq 0 \\ 0.001x, & x < 0 \end{cases} \quad (9)$$

$$\frac{d}{dx}\text{LeakyReLU}(x) = \begin{cases} 1, & x \geq 0 \\ 0.001, & x < 0 \end{cases} \quad (10)$$

III-F. Parametric ReLU

Permite aprender por un parámetro y varios pesos

$$\text{PReLU}(x) = \begin{cases} x, & x \geq 0 \\ ax, & x < 0 \end{cases} \quad (11)$$

$$\frac{d}{dx}\text{PReLU}(x) = \begin{cases} 1, & x \geq 0 \\ a, & x < 0 \end{cases} \quad (12)$$

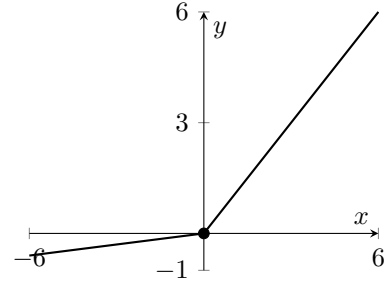


Figura 7. Función Leaky ReLU. Permite que la parte negativa tenga una pendiente

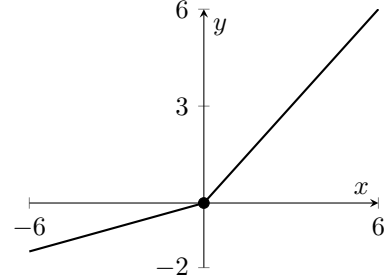


Figura 8. Función Parametric ReLU.

III-G. Soft Max

Es una función que cuenta con la característica de ser multiclase, genera una distribución de probabilidad cuya suma es 1 y la función de pérdida (Cross Entropy Loss) se usa para determinar qué tan diferente es la distribución predicha por el modelo que la real. Es usada como salida de clarificación, se usan vectores para crear una salida de probabilidades.

$$\text{Softmax}(z_i) = \frac{e^{z_i}}{\sum_{j=1}^K e^{z_j}} \quad (13)$$

IV. ¿CUÁL FUNCIÓN DE ACTIVACIÓN UTILIZAR?

Depende del requerimiento del modelo y la complejidad del problema.

La función ReLU es rápida de computar. Preferible empezar por esta función en cualquier modelo. Es especialmente utilizada en Deep Learning.

Leaky ReLU y Parametric ReLU requieren una mayor complejidad y, por lo tanto, un mayor conocimiento tanto del problema como del modelo a entrenar.

La función sigmoide y la tangente hiperbólica son funcionales pero presentan un mal gradiente por lo que no son recomendadas.

V. ¿QUÉ HACER PARA USAR UNA RED NEURONAL COMO REGRESIÓN?

$$wx + b$$

Básicamente se termina con una función lineal

VI. SOBRE EL PROYECTO

El proyecto tiene libertad de lenguaje en cuanto a la programación de la app móvil. La fecha de entrega estimada es el jueves 7 de mayo.

REFERENCIAS

- [1] Narodna Tribuna, "Convolutional Neural Networks," *Narodna Tribuna*. [Online]. Available: <https://narodnatribuna.info/lists/c/convolutional-neural-networks/>